

Tag 7

Fehler finden & die while-Schleife

Python-Grundlagen-Schulung

Was du heute lernst

1. Fehler **lesen, verstehen** und **reparieren** (Debugging)
2. Was ist eine **while-Schleife**?
3. Wie stoppe ich eine Schleife gezielt mit **break** und **continue**?
4. Viel Übung mit eigenen kleinen Programmen

Teil 2

Die while-Schleife

Warum brauchen wir Schleifen?

Willst du 4-mal "Ping gesendet" ausgeben,
kannst du die Zeile 4-mal abtippen.

Aber was, wenn es **400** oder **40.000** Mal sein sollen?

Oder du weißt vorher **gar nicht**, wie oft –

z. B.: "Frag so lange nach dem Passwort, bis es stimmt."

Genau dafür gibt es Schleifen.

Bild: Der Wachmann an der Schranke

Ein Wachmann hat eine einzige Regel:

"Solange noch Autos warten, lasse das nächste durch."

Er prüft das **nicht nur einmal**,
sondern vor **jedem einzelnen Auto neu**.

Erst wenn keine Autos mehr da sind, hört er auf.

Genau das macht while

```
zaehler = 1

while zaehler <= 3:
    print(f"Durchlauf Nummer: {zaehler}")
    zaehler = zaehler + 1
```

Vor **jedem** Durchlauf wird neu geprüft:

"Ist die Bedingung noch wahr?"

Schauen wir uns das Schritt für Schritt an →

Durchlauf 1

Ist `zaehler <= 3` ? `zaehler` ist **1** → `1 <= 3` → **Ja!**



Ausgabe: "Durchlauf Nummer: 1"



`zaehler` wird um 1 erhöht → jetzt **2**

Durchlauf 2

Ist `zaehler <= 3` ? `zaehler` ist **2** → `2 <= 3` → **Ja!**



Ausgabe: "Durchlauf Nummer: 2"



`zaehler` wird um 1 erhöht → jetzt **3**

Durchlauf 3

Ist `zaehler <= 3` ? `zaehler` ist **3** → `3 <= 3` → **Ja!**



Ausgabe: "Durchlauf Nummer: 3"



`zaehler` wird um 1 erhöht → jetzt **4**

Die Prüfung, die alles beendet

Ist `zaehler <= 3` ? `zaehler` ist `4` → `4 <= 3` → **Nein!**



Die Schleife stoppt.

Das Programm läuft direkt nach der Schleife weiter.

Der wichtigste Satz zur while-Schleife

Ohne `zaehler = zaehler + 1`

würde die Schleife NIE aufhören!

Diese Zeile ist **kein Extra** – sie ist der Grund, warum die Schleife irgendwann endet.

Achtung: Die Endlosschleife

Wenn die Bedingung **niemals** `False` wird,
läuft die Schleife **für immer**.

```
while True:  
    print("Ich laufe für immer!")
```

Keine Panik!

Passiert dir das aus Versehen in Colab:

☞ Klick einfach auf den **Stop-Button**
("Interrupt execution") neben der Zelle.

Das ist **kein Schaden** am System –
nur eine Berechnung, die du stoppen musst.

Ist while True: immer ein Fehler?

Nein! `while True:` ist ein bewusst genutztes, sehr gebräuchliches Muster.

Man nutzt es genau dann, wenn man die Schleife **von innen heraus** beenden möchte.

Wie das geht, lernst du jetzt: mit **break**!

Teil 3

break und continue

break: der Notausschalter

`break` beendet die Schleife **sofort und komplett** –
egal was die Bedingung am Kopf sagt.

Das Programm springt direkt zum Code **nach** der Schleife.

Beispiel: das unendliche Login

```
while True:
    eingabe = input("Passwort eingeben: ")
    if eingabe == "admin123":
        print("Schleife wird beendet.")
        break
    print("Falsches Passwort!")
```

`while True` heißt: "wiederhole für immer".

Aber die `if` -Bedingung **innerhalb** entscheidet,
wann `break` ausgelöst wird – und **dann** ist Schluss.

continue: der sanfte Sprung

`continue` beendet **nicht** die ganze Schleife –
nur den **aktuellen Durchlauf** wird abgebrochen,
die Schleife läuft danach ganz normal weiter.

Bild: Lied überspringen

`break` = die **ganze Playlist** stoppen 

`continue` = nur **diesen einen Song** überspringen,
die Playlist läuft weiter 

Beispiel: eine Zahl überspringen

```
zahl = 0
while zahl < 5:
    zahl = zahl + 1
    if zahl == 3:
        continue
    print(f"Zahl: {zahl}")
```

Ausgabe: 1, 2, 4, 5

Die 3 fehlt – bei ihr wurde `continue` ausgelöst,
und das `print` für diesen Durchlauf übersprungen.

Merkregel

break = Schleife komplett verlassen

**continue = diesen Durchlauf überspringen,
aber weitermachen**



Kurze Pause

Teil 4

Jetzt du: der große Übungsblock

Was dich erwartet

Kategorie A – Einfache Schleifen

Countdown, Fortschrittsbalken, Ping-Zähler

Kategorie B – mit break und continue

Login-Versuche, Ports überspringen, Chatbot

Kategorie C – Komplexere Aufgaben

Temperatur-Simulator, kleiner Task-Manager

Alle Aufgaben und Lösungen findest du im **Colab-Notebook**.

Ein Beispiel zum Warmwerden

```
counter = 5
while counter > 0:
    print(counter)
    counter = counter - 1

print("Patchvorgang startet...")
```

Ein einfacher Countdown – genau das Prinzip,
das du gerade Schritt für Schritt kennengelernt hast.

Was du heute gelernt hast

- ✓ Fehlermeldungen sind **Hinweise**, keine Katastrophen
- ✓ `while` prüft die Bedingung **vor jedem** Durchlauf neu
- ✓ `break` verlässt die Schleife komplett, `continue` überspringt nur einen Durchlauf
- ✓ Eine Endlosschleife ist mit dem Stop-Button jederzeit sicher zu stoppen

Morgen: Entscheidungen & Funktionen 🎉